Adaptive Replica Exchange Markov chain Monte Carlo for Job Shop Scheduling Problems

Rohith Ramanan[1,2]

Supervisor: Dr. Ramprasath S[3]

[1] Department of Physics, IIT Madras
[2] Wadhwani School of Data Science and AI, IIT Madras
[3] Department of Electrical Engineering, IIT Madras

Outline

Outline

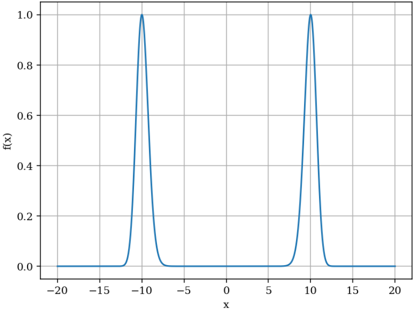
Problem Statement and Motivation

- Explore the use of Parallel Tempering for solving the NP-hard Job Shop Scheduling Problem — an approach that, to the best of our knowledge, has not been previously investigated.
- Integrate adaptive temperature tuning into the Parallel Tempering framework to dynamically adjust the temperature ladder. This enhances sampling efficiency and helps the algorithm escape local optima in complex solution landscapes.
- Design a method for generating valid neighboring solutions, along with an incremental strategy for updating the topological sort and recomputing longest paths. This enables faster traversal of the solution space while preserving feasibility.

Outline

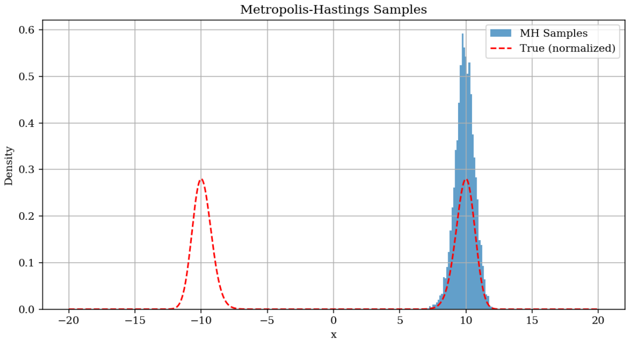
Sampling from Complex Distributions

- Want to sample from a **bimodal** target distribution:



- No closed-form **inverse CDF**
- **Normalization constant** is intractable
- Use Metropolis-Hastings:
  - Requires only the **unnormalized** target density

Can Metropolis-Hastings Handle Multimodal Distributions?



- Tends to get **stuck in one mode**—especially when modes are well separated
- In practice, many posteriors or energy landscapes are **highly multimodal**
- **Solution:** Use **Replica Exchange MCMC** (also known as **Parallel Tempering**)

Parallel Tempering

- Run **N parallel chains**, each using Metropolis-Hastings to sample from a **tempered** version of the target:

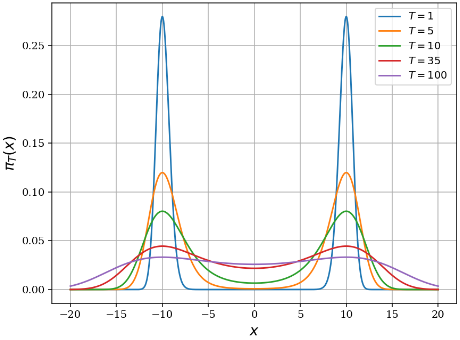
$$\pi^{\beta_i}(x) \propto \pi(x)^{\beta_i}$$

  where β_i is the **inverse temperature**
- The temperatures are ordered:

$$1 = \beta_1 > \beta_2 > \ldots > \beta_N > 0$$

- **Cold chain** ($\beta_1 = 1$) targets the desired distribution
- **Hot chains** (lower β) are more exploratory and easier to sample from
- **Occasional state swaps** between neighboring chains:
  - Allows information to flow between chains
  - Helps the cold chain **escape local minima**

Exploration vs. Tuning: The Trade-off in Parallel Tempering



- Shows tempered versions of a 1D distribution — **higher temperatures flatten the landscape**, allowing chains to move across modes easily
- This enables **exploration from any starting point**, even in highly multimodal settings
- **Limitation:** The effectiveness heavily depends on the **temperature ladder** — it often needs to be **tuned specifically** for each problem

Adaptivity in Parallel Tempering: Key Ideas

- Temperatures are updated dynamically to maximize the "flow of information" between chains, achieving optimal efficiency when the **swap acceptance probability (SAP)** is uniform across temperature pairs.
- It has been shown that the optimal SAP is approximately **23%** although values between 20% and 40% are generally acceptable Kone and Kofke 2005.
- In our approach, we parameterize the inverse temperatures using parameters $\rho_i^{(l)}$:

$$\beta_i^{(1)} \triangleq \frac{1}{T_{\text{low}}}, \quad \beta_i^{(l+1)} = \beta_i^{(l)} \cdot \sigma(\rho_i^{(l)}),$$

where $\sigma(z) = \dfrac{1}{1 + \exp(-z)}$ is the sigmoid function. Only $\beta^{(1)}$ is fixed; the rest are adapted via the $\rho_i^{(l)}$ parameters.

Adaptive Scheme: Details and Considerations

- The parameters $\rho_i^{(l)}$ are updated as:

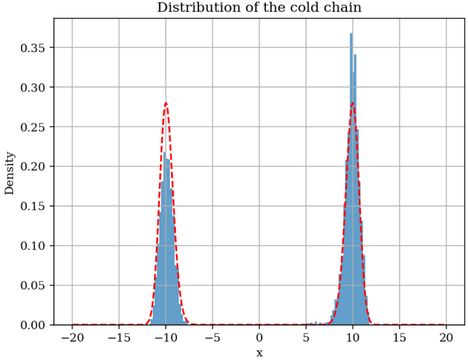
$$\rho_{i+1}^{(l)} = \Pi_d \left(\rho_i^{(l)} + \gamma(i) \cdot \left(\text{target\_sap} - \text{sap}_l\right) \right),$$

  where sap_i is the swap acceptance probability between chains i and $i+1$, and Π_d is a projection onto $[-d, \infty)$ with $d > 0$ to ensure numerical stability.
- The adaptation rate $\gamma(i)$ diminishes over iterations as:

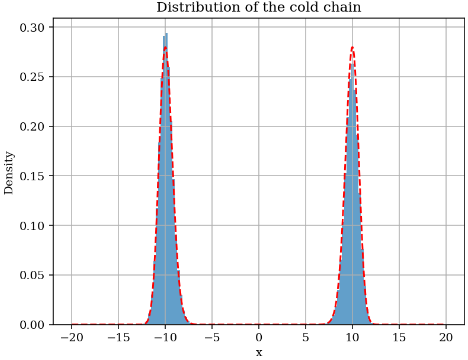
$$\gamma(i) = c \cdot (1+i)^{-\eta},$$

  following Roberts and Rosenthal 2007, who proved that such diminishing adaptation preserves ergodicity despite temporarily violating detailed balance — a key property ensuring convergence to the target distribution.

Experimental Results: Bimodal Distribution



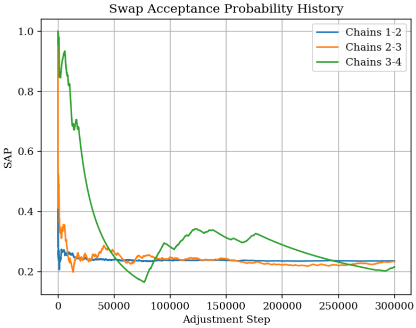
Vanilla Parallel Tempering



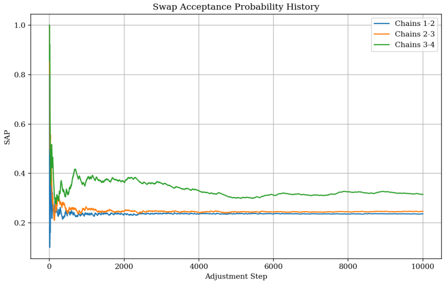
With Adaptive Parallel Tempering

We observe that Adaptive Parallel Tempering more accurately captures the desired bimodal distribution compared to the non-adaptive version.

Experimental Results: Convergence of SAP



Sampling from Bimodal Distribution



Solving 3-SAT Problem

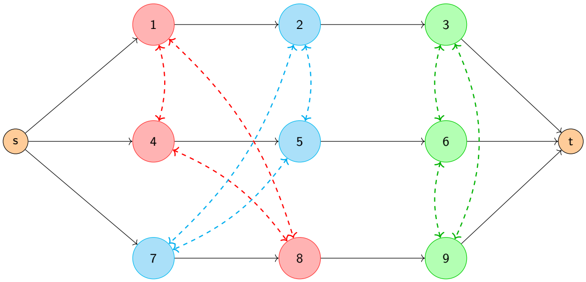
In the second plot, adaptive parallel tempering was used to solve a 3-SAT problem with 10 variables and 100 clauses.

Outline

Overview of the Job Shop Scheduling Problem

- $n \times m$ JSSP: We have n jobs with m machines, each job requiring m specific operations, each on a particular machine.
- Objective: Minimize the makespan — equivalent to assigning directions to undirected arcs in the *disjunctive graph* to minimize the length of the longest $s \to t$ path.

Job 1	(1,3)	(2,2)	(3,2)
Job 2	(1,2)	(2,1)	(3,4)
Job 3	(2,4)	(1,3)	(3,2)



Representation of a State

- A **state** is a partially ordered set (poset) represented by m chains, each corresponding to a machine's sequence of n operations:

$$L_D = \{(s_{1_1}, s_{1_2}, \ldots, s_{1_n}), \ldots, (s_{m_1}, s_{m_2}, \ldots, s_{m_n})\},$$

  where $(s_{i_1}, s_{i_2}, \ldots, s_{i_n})$ denotes the processing order on machine i.
- Let $L_C = \{(1, 2, \ldots, m), \ldots, ((n-1)m+1, \ldots, nm)\}$ represent the poset defined by job precedence constraints (conjunctive arcs).
- A state is **valid** if the union $L_C \cup L_D$ is also a poset. That is, it remains acyclic and transitively closed.
- Any linear extension π of this union corresponds to a valid topological sort.
- We can represent this with adjacency matrices: let

$$A = C + D,$$

  where C is the adjacency matrix of conjunctive arcs (excluding s and t), and D is that of disjunctive arcs. A valid permutation matrix encoding π will *simultaneously triangularize* both C and D.

Structure of C and D

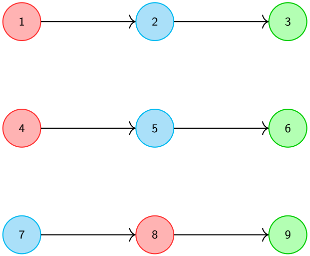


Figure: $\mathcal{G}_C$

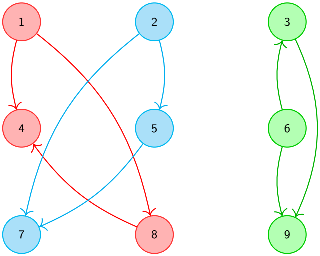
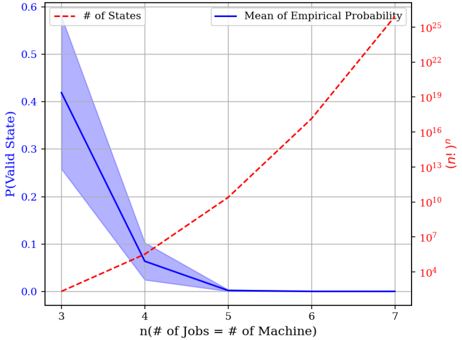


Figure: $\mathcal{G}_D$

- The graph $\mathcal{G}_C$ is a collection of linear chains for each job, while the graph $\mathcal{G}_D$ is a collection of transitive tournaments for each machine.
- π is the topological sorting of the whole graph $\mathcal{G}_{A+B} \iff P$ simultaneously triangularizes C and D.

The Curse of Consistency: Challenges in Neighbor Generation

- The size of the state space is $(n!)^m$, since each of the m machines in L_D can be independently ordered in $n!$ ways.
- However, the number of *valid* states—i.e., those where the union with $\mathcal{G}_C$ remains acyclic—is significantly smaller and decreases as the problem size increases.



Random permutations of operation sequences are highly inefficient. We need to propose a strategy that ensures feasibility with high probability.

Starting with L_D

- Suppose we begin with a feasible ordering L_D, and aim to reach another feasible ordering L'_D.
- Consider swapping two adjacent operations a and b on a machine in L_D.
- Since $\mathcal{G}_D$ is composed of transitive tournaments, such a swap flips a single directed edge between two nodes with no alternative paths between them. As a result, $\mathcal{G}_D$ remains acyclic.
- This updates the topological order π to a new order π' by the swapping the positions of a and b, which is still valid for $\mathcal{G}_D$, but may no longer be a valid topological order for $\mathcal{G}_C$.
- In other words, P' triangularizes D' but not C — i.e., it no longer simultaneously triangularizes both C and D.

Consistency with L_C

- In π', nodes a and b (with b before a) are operations on the same machine, and there are no other such operations between them.
- We can reorder the subarray $\pi'[\text{index}(b) : \text{index}(a)]$ so that b still precedes a, thus preserving consistency with L_D.
- Can this reordering help us recover a topological order π'' that is consistent with both L_D and L_C?
- Yes — if we can simultaneously triangularize the submatrices:

$$C'_{\text{sub}} = P'^T C P'[\pi'^{-1}(b) : \pi'^{-1}(a), \pi'^{-1}(b) : \pi'^{-1}(a)],$$

$$D'_{\text{sub}} = P'^T D P'[\pi'^{-1}(b) : \pi'^{-1}(a), \pi'^{-1}(b) : \pi'^{-1}(a)].$$

- This is possible if:

$$[C_{\text{sub}}, D'_{\text{sub}}] = 0 \quad \text{and} \quad [C_{\text{sub}}, D'_{\text{sub}}]D'_{\text{sub}} = 0,$$

  as shown in Shemesh 2016.
- These conditions hold trivially when there are no intermediate nodes, and can also be shown to hold when exactly one node lies between b and a.
- How can we exploit this property to generate valid neighbors efficiently?

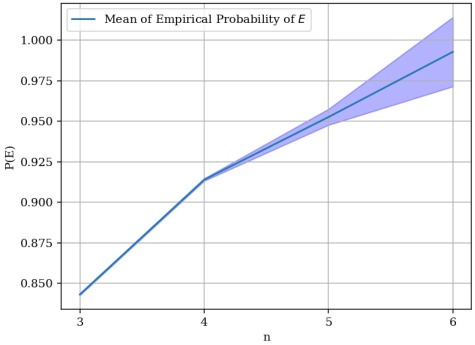
Structural Insight into the Search Space

- Let V be the set of all feasible disjunctive orderings L_D, and let $E \subset V$ be the subset containing at least one adjacent pair of operations (a, b) such that in a linear extension π of $L_C \cup L_D$, a and b occur with exactly one operation between them.
- If E is dense in V, then for any valid L_D, with high probability we can find such a pair (a, b), flip their order, and reorder the sublist $\pi[\text{index}(b) : \text{index}(a)]$ to construct a new valid L'_D.
- Although flipping adjacent nodes (with no node in between) is easier to implement, it often leads to cycling — revisiting the same few states.
- This motivates us to prioritize "almost-adjacent" node flips (with one node in between) to explore the state space more effectively.
- But this raises a critical question: **Is E dense in V?**

Key Observation for Valid Neighbor Generation

Key Observation

The probability that there exists at least one pair of adjacent operations in a valid L_D that appear with exactly one operation between them in a linear extension of $L_C \cup L_D$ **increases with the number of jobs and machines**.



Outline

Implementation Details

- The algorithm is implemented entirely in **C++** for high-performance execution.
- **OpenMP** is employed to parallelize the Metropolis-Hastings (MH) steps across multiple chains.
- Benchmarking is done using instances from the **Taillard Job Shop Scheduling Problem** dataset, a widely recognized benchmark in scheduling research.
- We compare our results with those obtained from **Google OR-Tools**, a state-of-the-art constraint solver for job shop scheduling.
- We ran both methods with a fixed computational budget of 5 minutes and recorded the best solution obtained. Additionally, we report the optimality gap between the obtained solution and the best-known lower bound for each instance.

Comparison with OR-Tools

Taillard Instance	AdapPT		Google-OR Tools	
	Solution	Optimality Gap %	Solution	Optimality Gap %
ta32 (30×15)	2011	9.2	1900	3.2
ta34 (30×15)	1960	5.8	1898	2.5
ta41 (30×20)	2277	10.3	2301	11.4
ta42 (30×20)	2101	5.9	2042	2.9
ta51 (50×15)	3458	25.3	3279	18.8
ta52 (50×15)	3325	20.6	3171	15.0
ta61 (50×20)	3156	10.0	3198	11.8
ta32 (50×20)	2921	6.0	3117	13.1
ta62 (100×20)	5496	6.0	5467	5.5
ta63 (100×20)	6057	8.7	5960	7.0

References I

Kone, Aminata and David A. Kofke (May 2005). "Selection of temperature intervals for parallel-tempering simulations". In: *The Journal of Chemical Physics* 122.20, p. 206101. ISSN: 0021-9606. DOI: 10.1063/1.1917749.

Roberts, Gareth O. and Jeffrey S. Rosenthal (2007). "Coupling and Ergodicity of Adaptive Markov Chain Monte Carlo Algorithms". In: *Journal of Applied Probability* 44.2, pp. 458–475. DOI: 10.1239/jap/1183667414.

Shemesh, Dan (2016). "A simultaneous triangularization result". In: *Linear Algebra and its Applications* 498. Special Issue: Legacy of Hans Schneider, pp. 394–398. ISSN: 0024-3795. DOI: https://doi.org/10.1016/j.laa.2015.09.044. URL: https://www.sciencedirect.com/science/article/pii/S0024379515005716.